

ICDFA INDIVIDUAL PROJECT REPORT

TCP vs UDP

COMMUNICATION FAILURE & TROUBLESHOOTING

A packet-level study of a secure website connection (x.com)

STUDENT NAME	Samina Parveen
ICDFA REG. NUMBER	C11/26/FCDF/17139
Fellowship Program	ICDFA Fellowship in Cybersecurity & Digital Forensics
TEAM / COHORT	TEAM-08 · FCDF-COHORT-11
LEVEL / SEMESTER	Level 1 · Semester 1
PROJECT	TCP IP Communication in Action: Group Presentation
SELECTED SCENARIO	Opening a secure website (x.com over HTTPS)
INDIVIDUAL TASK	TCP vs UDP · Application Examples · Communication Failure · Troubleshooting Procedure
REPORT DATE	30 September 2026
GROUP DEADLINE	02 October 2026, 11:59 PM

Evidence captured with Wireshark 4.4.15, Windows command line and PowerShell

Contents

1	Introduction and my role	Purpose, scope, test environment
2	TCP vs UDP	Comparison table and packet-level evidence
3	Application examples	Which protocol suits which application
4	Communication failure	DNS failure and blocked port scenarios
5	Troubleshooting procedure	Step-by-step diagnosis, fix and verification
6	Conclusion	Key findings and lessons learned
7	References and declarations	Sources, AI-use and contribution declaration

Executive summary

This report covers my part of the TEAM-08 group presentation on how information travels across a network using the TCP/IP model. The team selected the scenario "opening a secure website". I was responsible for four items: the TCP vs UDP comparison, application examples, one realistic communication failure, and the troubleshooting procedure.

All findings are backed by my own packet captures and command-line tests. The capture of a real HTTPS visit to x.com shows DNS using UDP port 53 and the web session using TCP port 443, including the three-way handshake and the TLS Client Hello. I then reproduced a DNS failure by pointing the PC at an unreachable DNS server (192.0.2.1). Pinging 8.8.8.8 still worked while name resolution failed. A second test showed TCP port 444 silently blocked, with the SYN retransmitted and never answered. Both faults were located layer by layer, fixed and verified.

Key takeaways

TCP gives reliable, ordered delivery and is used for the web session; UDP gives low-overhead, connectionless delivery and is used for DNS lookups. A failure in one protocol (DNS over UDP) can make the whole website appear "down" even when TCP and IP connectivity are perfectly healthy.

Item	Detail
My role	TCP vs UDP comparison, application examples, communication failure, troubleshooting procedure
Tools used	Wireshark 4.4.15, ipconfig, ping, nslookup, tracert, curl, Test-NetConnection, PowerShell
Evidence	36 evidence figures (sets A to E): my original screenshots (Figures 6 to 36) and clean re-created panels for the setup screens (Figures 1 to 5); capture file integrity hash recorded (Figure 5)

1 INTRODUCTION AND MY ROLE

The group project asks us, as junior network analysts, to trace one communication scenario through the TCP/IP model. Our team chose opening a secure website. My individual contribution focuses on the transport layer (TCP and UDP) and on what happens when communication fails, together with the method a network administrator would use to find and fix the fault.

1.1 Test environment

Component	Value observed
Client PC	Windows, Wi-Fi adapter, IPv4 192.168.1.25
Default gateway	192.168.1.1 (home router)
DNS server (normal)	192.168.1.1 (router forwards queries)
Destination	x.com, HTTPS on TCP port 443 (resolved to 172.66.0.227 in the capture)
Capture tool	Wireshark 4.4.15 with capture filter "port 53 or port 443" on the Wi-Fi interface

Note: the name x.com is served from a global CDN, so different resolvers and moments return different addresses (172.66.0.227 in the capture and in the final nslookup, 162.159.140.229 in the baseline and DNS-fix tests). Both belong to the same hosting network and do not affect the analysis.

1.2 Baseline evidence

Before capturing, I recorded the IP configuration, cleared the DNS and ARP caches so that every step of the lookup would appear in the capture, made a verbose HTTPS request with curl, and hashed the capture file to preserve its integrity.

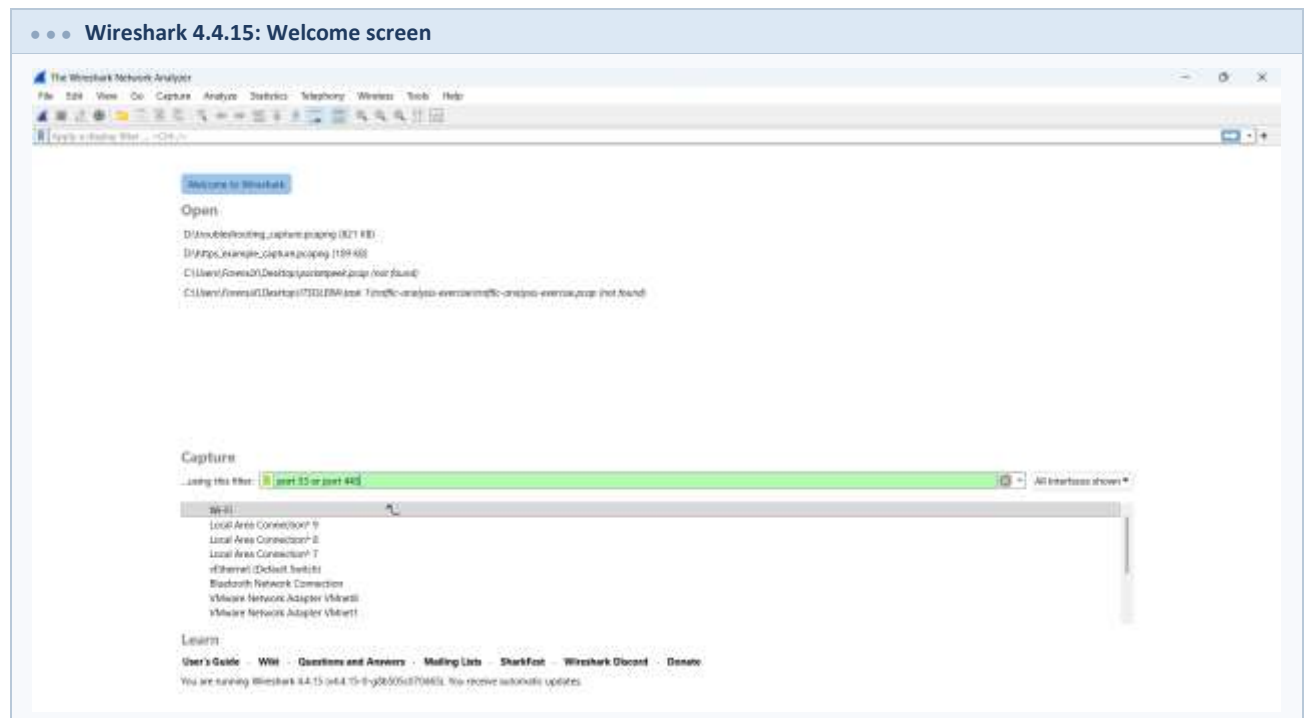


Figure 1 (A0): Wireshark capture setup (re-created as text): Wi-Fi interface with a capture filter limited to DNS and HTTPS traffic

... Command Prompt: ipconfig /all

Wireless LAN adapter Wi-Fi:

```

Description . . . . . : Intel(R) Wi-Fi 6 AX201 160MHz
Physical Address. . . . . : 2C-DB-07-41-7E-95
DHCP Enabled. . . . . : Yes
IPv4 Address. . . . . : 192.168.1.25(Preferred)
Subnet Mask . . . . . : 255.255.255.0
Lease Obtained. . . . . : Wednesday, September 30, 2026 7:14:26 PM
Default Gateway . . . . . : fe80::1000%9
                             192.168.1.1
DHCP Server . . . . . : 192.168.1.1
DNS Servers . . . . . : 192.168.1.1

```

Figure 2 (A1): ipconfig /all: IPv4 192.168.1.25, gateway 192.168.1.1, DNS 192.168.1.1

... Command Prompt: clear DNS and ARP caches

```

C:\Windows\System32>ipconfig /flushdns
Windows IP Configuration
Successfully flushed the DNS Resolver Cache.

C:\Windows\System32>arp -d *

```

Figure 3 (A2): ipconfig /flushdns and arp -d both completed successfully

... Command Prompt: curl verbose HTTPS request

```

C:\Windows\System32>curl.exe -v --http1.1 https://x.com -o NUL
* Host x.com:443 was resolved.
* IPv6: (none)
* IPv4: 172.66.0.227
  Trying 172.66.0.227:443...
* ALPN: curl offers http/1.1
* ALPN: server accepted http/1.1
* Established connection to x.com (172.66.0.227 port 443) from 192.168.1.25 port 55125
> GET / HTTP/1.1
> Host: x.com
> User-Agent: curl/8.21.0
< HTTP/1.1 200 OK
< Date: Wed, 30 Sep 2026 16:38:59 GMT
< Content-Type: text/html; charset=UTF-8
< Server: cloudflare envoy

```

Figure 4 (A3): curl verbose (--http1.1) to x.com on port 443, result HTTP 200 OK

... Command Prompt: capture file integrity hash

```

C:\Windows\System32>certutil -hashfile "%USERPROFILE%\Desktop\x_capture.pcapng" SHA256
SHA256 hash of C:\Users\ForensiX\Desktop\x_capture.pcapng:
c23a7cb6022ce8bd0236dd8cfb854689be289c33cce2bbdbd179b1ea3b005d81
CertUtil: -hashfile command completed successfully.

```

Figure 5 (A4): SHA-256 hash of x_capture.pcapng, recorded as evidence of integrity (re-created as text)

2 TCP VS UDP

Both protocols operate at the Transport layer of the TCP/IP model (Layer 4 of OSI) and use port numbers to deliver data to the correct application. They make very different trade-offs between reliability and speed.

2.1 Comparison table

Feature	TCP	UDP
Full name	Transmission Control Protocol (RFC 9293)	User Datagram Protocol (RFC 768)
Connection	Connection-oriented: three-way handshake before data	Connectionless: sends immediately
Reliability	Acknowledgements and retransmission of lost data	No acknowledgement, no retransmission
Ordering	Sequence numbers keep data in order	No ordering; datagrams may arrive out of order
Flow / congestion control	Yes (window size, congestion control)	None built in
Header size	20 bytes minimum (up to 60 with options)	8 bytes (source port, destination port, length, checksum)
Data unit (PDU)	Segment	Datagram
Overhead and speed	Higher overhead, slower start-up	Low overhead, low latency
Best for	Accuracy matters more than speed	Speed matters more than perfect delivery
In our scenario	HTTPS session to x.com on port 443	DNS query to 192.168.1.1 on port 53

2.2 UDP in the capture: the DNS lookup

To open x.com the browser first needs an IP address. The PC sent a DNS query as a single UDP datagram to the router. The response returned the A record 172.66.0.227 with no IPv6 (AAAA) answer. There was no handshake: the query simply went out and the reply came back.

No.	Time	Source	Destination	Protocol	Length	Info
11	2.537005	192.168.1.25	192.168.1.1	DNS	65	Standard query 0x0002 A x.com
12	2.562335	192.168.1.1	192.168.1.25	DNS	81	Standard query response 0x0002 A x.com A 172.66.0.227
13	2.565558	192.168.1.25	192.168.1.1	DNS	65	Standard query 0x0003 AAAA x.com
14	2.602661	192.168.1.1	192.168.1.25	DNS	133	Standard query response 0x0003 AAAA x.com SOA a.u10.tutodes.net

Figure 6 (B1): DNS query and response for x.com (A record 172.66.0.227, AAAA: no IPv6 record)

```

> Frame 11: 65 bytes on wire (520 bits), 65 bytes captured (520 bits) on interface \Device\NPF_{65089B2D-BA1C-430B-9438-FF3665A82D5A}, id 0
> Ethernet II, Src: Intel_41:7e:95 (2c:db:07:41:7e:95), Dst: FiberhomeTel_94:70:a0 (ec:e6:a2:94:70:a0)
> Internet Protocol Version 4, Src: 192.168.1.25, Dst: 192.168.1.1
> User Datagram Protocol, Src Port: 57452, Dst Port: 53
  Source Port: 57452
  Destination Port: 53
  Length: 31
  Checksum: 0x839b [unverified]
  [Checksum Status: Unverified]
  [Stream index: 3]
  [Stream Packet Number: 1]
  [Timestamps]
    [Time since first frame: 0.000000000 seconds]
    [Time since previous frame: 0.000000000 seconds]
  UDP payload (23 bytes)
  Domain Name System (query)
    Transaction ID: 0x0002
    > Flags: 0x0100 Standard query
    Questions: 1
    Answer RRs: 0
    Authority RRs: 0
    Additional RRs: 0
    > Queries
      [Response In: 12]

```

Figure 7 (B2): UDP header of the DNS query: source port 57452, destination port 53, length 31

What the UDP header tells us

The length field of 31 bytes is an 8-byte UDP header plus a 23-byte DNS question. The source port (57452) is an ephemeral port chosen by the PC; the destination port 53 identifies the DNS service. The header holds only four fields, which is why UDP is so lightweight.

```

> Frame 12: 81 bytes on wire (648 bits), 81 bytes captured (648 bits) on interface \Device\NPF_{65089B2D-BA1C-430B-9438-FF3665A82D5A}, id 0
> Ethernet II, Src: FiberhomeTel_94:70:a0 (ec:e6:a2:94:70:a0), Dst: Intel_41:7e:95 (2c:db:07:41:7e:95)
> Internet Protocol Version 4, Src: 192.168.1.1, Dst: 192.168.1.25
> User Datagram Protocol, Src Port: 53, Dst Port: 57452
  Domain Name System (response)
    Transaction ID: 0x0002
    > Flags: 0x8100 Standard query response, No error
    Questions: 1
    Answer RRs: 1
    Authority RRs: 0
    Additional RRs: 0
    > Queries
    > Answers
      x.com: type A, class IN, addr 172.66.0.227
        Name: x.com
        Type: A (1) (Host Address)
        Class: IN (0x0001)
        Time to live: 147 (2 minutes, 27 seconds)
        Data length: 4
        Address: 172.66.0.227
      [Request in: 11]
      [Time: 0.035270000 seconds]

```

Figure 8 (B3): DNS answer: x.com A record, IPv4 address 172.66.0.227, TTL 147 seconds

2.3 TCP in the capture: connection, security and close

With the address known, the PC opened a TCP connection to 172.66.0.227 on port 443. The three-way handshake (SYN, SYN-ACK, ACK) established the session and initial sequence numbers. Immediately afterwards the client sent a TLS Client Hello naming the site (SNI = x.com). When finished, both sides closed the connection with a FIN/ACK exchange.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	192.168.1.1	192.168.1.1	TCP	60	80000 → 10 [RST] Seq=1000000000, Len=0 MSS=100, SACK_PERM=
2	0.000119	192.168.1.1	192.168.1.1	TCP	60	57330 → 10 [RST] Seq=1000000000, Len=0 MSS=100, SACK_PERM=
3	0.000313	192.168.1.1	192.168.1.1	TCP	60	56800 → 10 [RST] Seq=1000000000, Len=0 MSS=100, SACK_PERM=
4	0.001703	192.168.1.1	192.168.1.1	TCP	60	94000 → 10 [RST] Seq=1000000000, Len=0 MSS=100, SACK_PERM=
5	0.001700	192.168.1.1	192.168.1.1	TCP	60	99500 → 10 [RST] Seq=1000000000, Len=0 MSS=100, SACK_PERM=
6	0.001700	192.168.1.1	192.168.1.1	TCP	60	80000 → 10 [RST] Seq=1000000000, Len=0 MSS=100, SACK_PERM=
13	0.500037	192.168.1.1	192.168.1.1	TCP	60	TCP Port numbers mismatch: 52000 → 85 [RST] Seq=1000000000, Len=0 MSS=100, SACK_PERM=
14	0.500133	192.168.1.1	192.168.1.1	TCP	60	TCP Port numbers mismatch: 53310 → 85 [RST] Seq=1000000000, Len=0 MSS=100, SACK_PERM=
15	0.500363	192.168.1.1	192.168.1.1	TCP	60	TCP Port numbers mismatch: 56800 → 85 [RST] Seq=1000000000, Len=0 MSS=100, SACK_PERM=
17	0.501347	192.168.1.1	0.0.0.0	TCP	60	53310 → 403 [RST] Seq=1000000000, Len=0 MSS=100, SACK_PERM=
18	0.502509	2000-ach-7-1100-99000	2000-ach-7-1100-99000	TCP	60	49300 → 403 [RST] Seq=1000000000, Len=0 MSS=100, SACK_PERM=
118	7.870844	192.168.1.1	10.107.161.177	TCP	60	42411 → 403 [RST] Seq=1000000000, Len=0 MSS=100, SACK_PERM=
119	7.180542	192.168.1.1	10.108.160.65	TCP	60	58000 → 403 [RST] Seq=1000000000, Len=0 MSS=100, SACK_PERM=
119	7.200576	192.168.1.1	10.107.161.177	TCP	60	90000 → 403 [RST] Seq=1000000000, Len=0 MSS=100, SACK_PERM=
119	8.077793	192.168.1.1	10.108.160.65	TCP	60	51120 → 403 [RST] Seq=1000000000, Len=0 MSS=100, SACK_PERM=
119	8.101120	192.168.1.1	10.108.160.65	TCP	60	51120 → 403 [RST] Seq=1000000000, Len=0 MSS=100, SACK_PERM=

Figure 9 (B4): TCP SYN packet from the PC to 172.66.0.227:443 (flags 0x002)

```

1 Frame 171: 66 bytes on wire (528 bits), 66 bytes captured (528 bits) on interface \\Device\\NPF{B5089B2D-8A1C-9438-FF1655A82D5A}, id 0
2 Ethernet II, Src: Intel_41:7c:95 (2c:d0:07:41:7c:95), Dst: FiberhomeTel_94:70:a0 (ec:a0:a2:94:70:a0)
3 Internet Protocol Version 4, Src: 192.168.1.25, Dst: 172.16.0.227
4 Transmission Control Protocol, Src Port: 55125, Dst Port: 443, Seq: 0, Len: 0
    Source Port: 55125
    Destination Port: 443
    [Stream index: 22]
    [Stream Packet Number: 1]
    [Conversation completeness: Complete, WITH_DATA (31)]
    [TCP Segment Len: 0]
    Sequence Number: 0 (relative sequence number)
    Sequence Number (raw): 1750609356
    [Next Sequence Number: 1 (relative sequence number)]
    Acknowledgment number: 0
    Acknowledgment number (raw): 0
    [Window length: 32 bytes (8)]
    Flags: 0x002 (SYN)
    Window: 65535
    [Calculated window size: 65535]
    Checksum: 8x6F0d [unverified]
    [Checksum Status: Unverified]
    Urgent Pointer: 0
    Options: (12 bytes), Maximum segment size, No-Operation (NOP), Window scale, No-Operation (NOP), No-Operation (NOP), SACK permitted
    [...more options]

```

Figure 10 (B5): TCP three-way handshake with x.com (SYN, SYN-ACK, ACK)

No.	Time	Source	Destination	Protocol	Length	Info
85	0.332635	172.66.0.227	192.168.1.25	TCP	60	443 → 80 [ACK] Seq=813633333 Win=1 Len=0
171	18.181257	192.168.1.25	172.66.0.227	TCP	60	80 → 443 [SPR] Seq=813633333 Len=0 MSS=1460 Win=256 SACK_PERM
172	18.200366	172.66.0.227	192.168.1.25	TCP	60	443 → 55125 [SPR, ACK] Seq=813633333 Win=55125 Len=0 MSS=1460 SACK_PERM Win=55125
173	18.208472	192.168.1.25	172.66.0.227	TCP	54	55125 → 443 [ACK] Seq=813633333 Len=0
174	18.21834	192.168.1.25	172.66.0.227	TLSv1.3	508	[Client Hello] [SRV=www]
175	18.227421	172.66.0.227	192.168.1.25	TCP	60	443 → 55125 [ACK] Seq=813633333 Win=11072 Len=0
176	18.235166	172.66.0.227	192.168.1.25	TLSv1.3	1460	Server Hello, Change Cipher Spec

Figure 11 (B5): Handshake followed by the TLS Client Hello (SNI = x.com)

```

> Frame 172: 66 bytes on wire (528 bits), 66 bytes captured (528 bits) on interface \Device\NPF_{658B982D-9A1C-430B-943B-F5656A82D5A}, Id 0
> Ethernet II, Src: FiberhomeTel-94:70:a0 (ac:a0:a2:94:70:a0), Dst: Intel_41:7a:95 (2c:cd:87:41:7a:95)
> Internet Protocol Version 4, Src: 172.66.0.227, Dst: 192.168.1.25
> Transmission Control Protocol, Src Port: 443, Dst Port: 55125, Seq: 8, Ack: 1, Len: 0
  Source Port: 443
  Destination Port: 55125
  [Stream Index: 22]
  [Stream Packet Number: 2]
  [Conversation completeness: Complete, WITH_DATA (31)]
  [TCP Segment Len: 0]
  Sequence Number: 8 (relative sequence number)
  Sequence Number (raw): 3014531262
  [Next Sequence Number: 1 (relative sequence number)]
  Acknowledgment Number: 1 (relative ack number)
  Acknowledgment number (raw): 1758689357
  1000 ..... = Header length: 32 bytes (8)
  [Flags: 0x012 (SYN, ACK)]
  Window: 65535
  [Calculated window size: 65535]
  Checksum: 8xc4d6 [unverified]
  [Checksum Status: Unverified]
  Urgent Pointer: 0
  Options: (12 bytes), Maximum segment size, No-Operation (NOP), No-Operation (NOP), SACK permitted, No-Operation (NOP), Window scale
  [Timestamps]
  [SEQ/ACK analysis]

```

Figure 12 (B6): SYN-ACK from x.com: source port 443, destination port 55125, flags 0x012 (SYN + ACK)

No.	Time	Source	Destination	Protocol	Length	Info
26	0.248514	192.168.1.25	4.207.44.68	TLSv1.3	9907	Client Hello [SSL-tls-teams.events.data.microsoft.com]
28	0.390379	192.168.1.25	4.207.44.68	TLSv1.3	843	Change Cipher Spec, Client Hello [SSL-tls-teams.events.data.microsoft.com]
62	2.223130	2400:adcf:3300:9906::	2001:486b:03b3:7560::	TLSv1.3	2628	Client Hello [UNL-name-google.fr.com]
110	7.458020	192.168.1.25	32.187.161.177	TLSv1.2	2304	Client Hello [UNL-diagnostic-prod-us-west-2.amazonaws.com]
138	7.731605	192.168.1.25	32.187.161.177	TLSv1.2	2340	Client Hello [UNL-diagnostic-prod-us-west-2.amazonaws.com]
174	38.222814	192.168.1.25	172.66.0.227	TLSv1.3	500	Client Hello [SSL-tls.com]

Figure 13 (B7): TLS Client Hello packets; packet 174 shows SNI = x.com

[illegible]

Figure 14 (B7): TLS Client Hello expanded: SNI = x.com; version field 0x0303 (TLS 1.2 legacy), supported_versions offers TLS 1.3 and 1.2

No.	Time	Source	Destination	Protocol	Length	Info
42	1.208205	192.168.1.25	172.16.0.227	TCP	54	33923 → 443 [RST, ACK] Seq=1 Ack=0 Win=256 Len=0
43	1.306753	172.16.0.227	192.168.1.25	TCP	60	443 → 43023 [FIN, ACK] Seq=48 Ack=1 Win=17 Len=0
223	10.729556	192.168.1.25	172.16.0.227	TCP	54	55125 → 443 [FIN, ACK] Seq=648 Ack=48555 Win=65240 Len=0
224	10.753685	172.16.0.227	192.168.1.25	TCP	60	443 → 55125 [FIN, ACK] Seq=48555 Ack=649 Win=131072 Len=0

Figure 15 (B8): TCP connection close with x.com (FIN/ACK exchange on port 55125)

2.4 Traffic share and conversations

Wireshark statistics confirm the split of roles: TCP carries the bulk of the traffic (HTTPS with TLS), while UDP carries only the small DNS exchanges. The conversation tables show one large TCP conversation to port 443 and three short UDP DNS conversations to port 53.

Protocol	Percent Packets	Packets	Percent Bytes	Bytes	Bits/s	End Packets	End Bytes	End Bits/s	PDUs
▼ Frame	100.0	386	100.0	121880	41 k	0	0	0	386
▼ Ethernet	100.0	386	4.6	5626	1928	0	0	0	386
▼ Internet Protocol Version 6	38.6	149	4.9	5960	2042	0	0	0	149
▼ Transmission Control Protocol	38.6	149	2.5	3052	1046	118	2432	833	149
Transport Layer Security	8.0	31	6.8	8333	2856	31	8333	2856	31
▼ Internet Protocol Version 4	61.4	237	3.9	4740	1624	0	0	0	237
▼ User Datagram Protocol	3.1	12	0.1	96	32	0	0	0	12
Domain Name System	3.1	12	0.9	1150	394	12	1150	394	12
▼ Transmission Control Protocol	58.3	225	4.1	5028	1723	139	3308	1133	225
Transport Layer Security	22.3	86	73.6	87243	29 k	86	85596	29 k	89

Figure 16 (B9): Protocol Hierarchy: TCP 58.3% (HTTPS/TLS), UDP 3.1% (DNS), TLS 22.3%

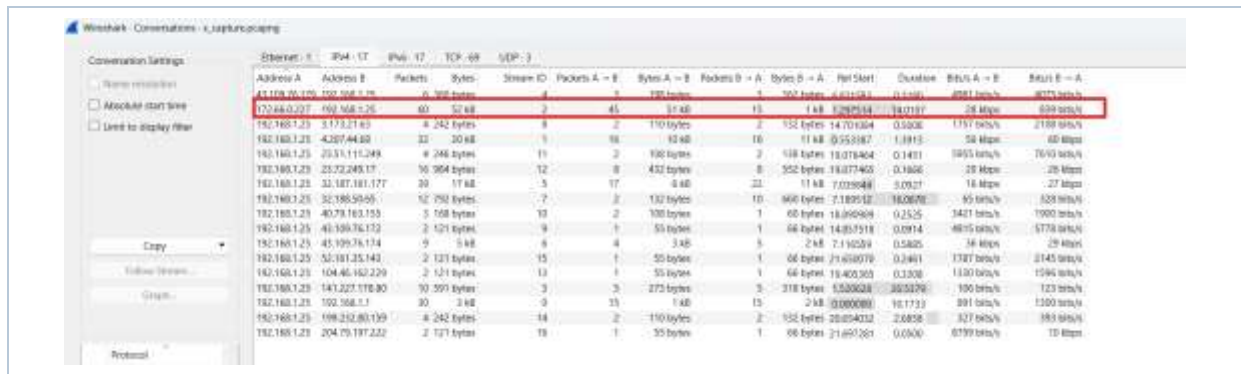


Figure 17 (B10): IPv4 conversations: x.com 172.66.0.227 ↔ PC, 60 packets, 52 kB, about 14 seconds

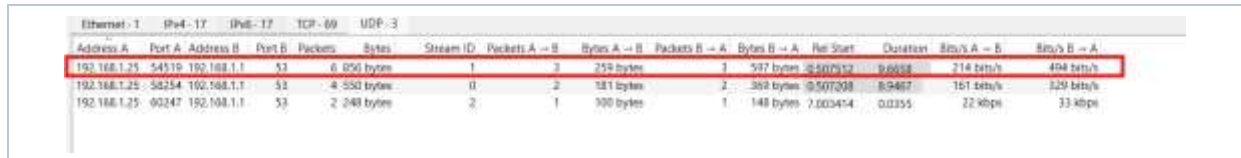


Figure 18 (B10): UDP conversations: DNS to 192.168.1.1 on port 53, source ports 54519, 58254 and 60247

3 APPLICATION EXAMPLES

The right transport protocol depends on what the application can tolerate. Lost data that must be perfect calls for TCP; data that is useless if it arrives late calls for UDP.

TCP: reliability first	UDP: speed first
Secure web browsing (HTTPS, port 443). Every byte of a page must arrive intact and in order. Our capture shows exactly this.	DNS lookups (port 53). One small question and one small answer; a handshake would cost more than the data itself. Seen in Figure 7.
Email (SMTP 25/587, IMAP 143/993). Messages must not be corrupted or lost.	Video meetings and VoIP (RTP). A late audio packet is worthless; a brief glitch is better than a delay.
File transfer (FTP 20/21, SFTP/SSH 22). A single missing block corrupts the file.	Live streaming and online gaming. Current data matters more than retransmitting old data.
Remote administration (SSH, RDP). Commands and screen updates must be reliable.	DHCP and NTP. Short request and reply exchanges where simplicity wins.

Both protocols in one website visit

Opening a secure website uses both: UDP for the DNS lookup, then TCP for the HTTPS session. Newer web traffic using HTTP/3 (QUIC) runs over UDP port 443 and rebuilds reliability in the application layer. My test forced HTTP/1.1 (curl --http1.1), so the capture shows the classic TCP path.

4 COMMUNICATION FAILURE

I reproduced two realistic failures in the secure-website scenario. The main one is a DNS failure; the second is a blocked TCP port. They show faults at different points of the UDP/TCP story above.

4.1 Main failure: website unreachable because DNS does not respond

Failure summary

Symptom: x.com cannot be opened; tools report "could not find host" / "Could not resolve host".

Root cause: the PC's DNS server was set to 192.0.2.1, a reserved address on which nothing answers.

Affected layer: Application (DNS over UDP port 53). Network connectivity itself was healthy.

Baseline. Before the fault, ping x.com succeeded (4/4 replies) and nslookup returned 162.159.140.229.

```
C:\Windows\System32>ping x.com -n 4

Pinging x.com [162.159.140.229] with 32 bytes of data:
Reply from 162.159.140.229: bytes=32 time=23ms TTL=57
Reply from 162.159.140.229: bytes=32 time=15ms TTL=57
Reply from 162.159.140.229: bytes=32 time=15ms TTL=57
Reply from 162.159.140.229: bytes=32 time=15ms TTL=57

Ping statistics for 162.159.140.229:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 15ms, Maximum = 23ms, Average = 17ms

C:\Windows\System32>nslookup x.com
Server: UnKnown
Address: 192.168.1.1

Non-authoritative answer:
Name:     x.com
Address:  162.159.140.229
```

Figure 19 (C1): Baseline: ping x.com (4/4 replies) and nslookup x.com (162.159.140.229) working

Fault introduced. I changed the DNS server in PowerShell to 192.0.2.1 and flushed the cache. The resulting errors:

```
PS C:\WINDOWS\system32> Set-DnsClientServerAddress -InterfaceAlias "Wi-Fi" -ServerAddresses 192.0.2.1
PS C:\WINDOWS\system32> ipconfig /flushdns

Windows IP Configuration

Successfully flushed the DNS Resolver Cache.
PS C:\WINDOWS\system32>
```

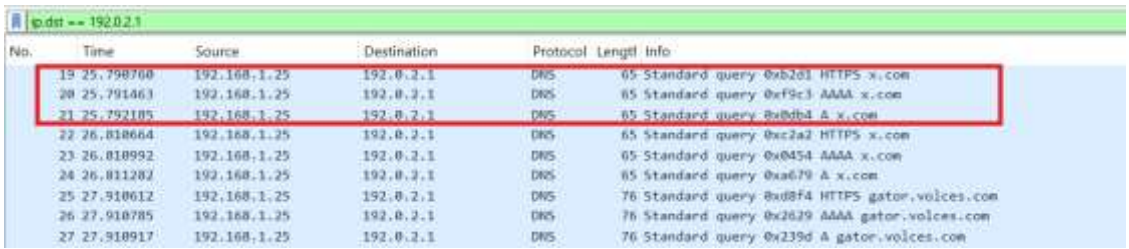
Figure 20 (C2b): PowerShell: DNS server changed to 192.0.2.1 (reserved address, nothing answers), cache flushed

```
C:\Windows\System32>ping x.com -n 2
Ping request could not find host x.com. Please check the name and try again.

C:\Windows\System32>curl.exe -I https://x.com
curl: (6) Could not resolve host: x.com
```

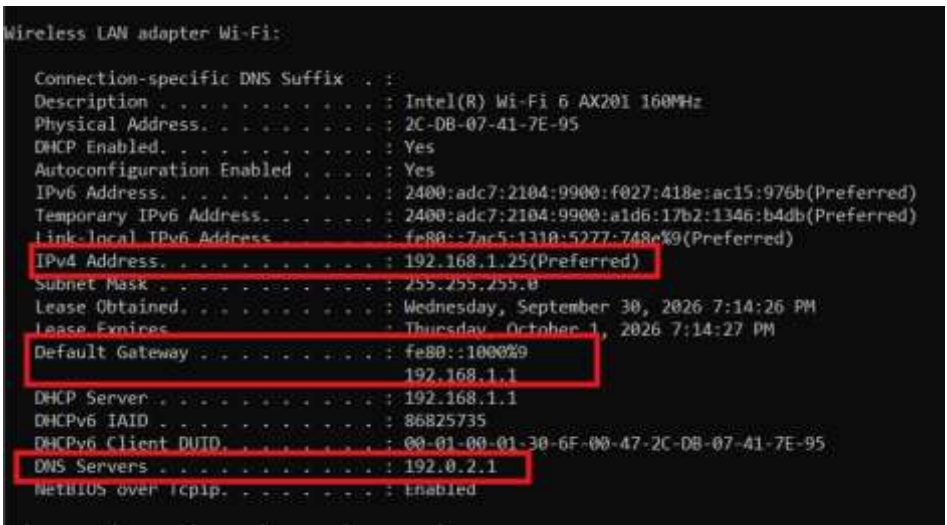
Figure 21 (C2): DNS failure symptoms: "could not find host" and "Could not resolve host"

What the capture shows. Wireshark (filter `ip.dst == 192.0.2.1`) shows the PC sending DNS queries for `x.com` (HTTPS, AAAA and A records) to `192.0.2.1`. No reply ever arrives, and the same three queries repeat roughly every second. This is the UDP trade-off in action: UDP has no handshake and no built-in retransmission, so the application (the DNS resolver) must time out and retry on its own.



No.	Time	Source	Destination	Protocol	Length	Info
19	25.798700	192.168.1.25	192.0.2.1	DNS	65	Standard query 0xb2d1 HTTPS x.com
20	25.791463	192.168.1.25	192.0.2.1	DNS	65	Standard query 0xf9c3 AAAA x.com
21	25.792185	192.168.1.25	192.0.2.1	DNS	65	Standard query 0xbdb4 A x.com
22	26.818664	192.168.1.25	192.0.2.1	DNS	65	Standard query 0xc2a2 HTTPS x.com
23	26.818992	192.168.1.25	192.0.2.1	DNS	65	Standard query 0x0454 AAAA x.com
24	26.811282	192.168.1.25	192.0.2.1	DNS	65	Standard query 0xae79 A x.com
25	27.910612	192.168.1.25	192.0.2.1	DNS	76	Standard query 0xd8f4 HTTPS gator.voices.com
26	27.910785	192.168.1.25	192.0.2.1	DNS	76	Standard query 0xc2629 AAAA gator.voices.com
27	27.910917	192.168.1.25	192.0.2.1	DNS	76	Standard query 0x239d A gator.voices.com

Figure 22 (C3): DNS queries to `192.0.2.1` with no response. Packets 19 to 21 show repeated queries for `x.com` with no reply



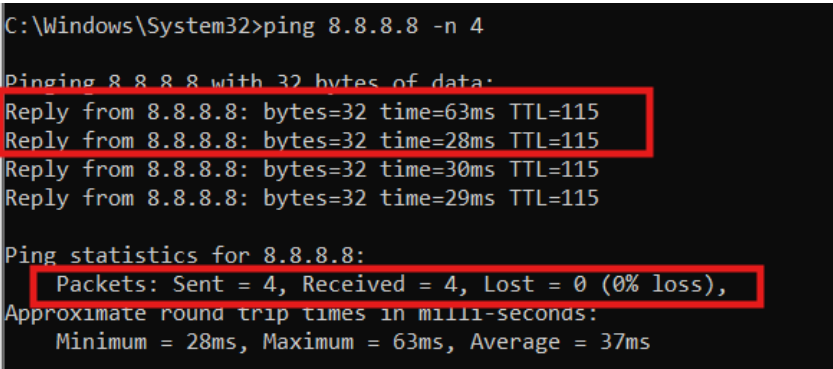
```

Wireless LAN adapter Wi-Fi:

    Connection-specific DNS Suffix . . . : 
    Description . . . . . : Intel(R) Wi-Fi 6 AX201 160MHz
    Physical Address. . . . . : 2C-08-07-41-7E-95
    DHCP Enabled. . . . . : Yes
    Autoconfiguration Enabled . . . . : Yes
    IPv6 Address. . . . . : 2400:adc7:2104:9900:f027:418e:ac15:976b(Preferred)
    Temporary IPv6 Address. . . . . : 2400:adc7:2104:9900:a1d6:17b2:1346:b4db(Preferred)
    Link-local IPv6 Address . . . . . : fe80::7nr5:1318:5277:748c%9(Preferred)
    IPv4 Address. . . . . : 192.168.1.25(Preferred)
    Subnet Mask . . . . . : 255.255.255.0
    Lease Obtained. . . . . : Wednesday, September 30, 2026 7:14:26 PM
    Lease Expires . . . . . : Thursday, October 1, 2026 7:14:27 PM
    Default Gateway . . . . . : fe80::1000%9
    . . . . . : 192.168.1.1
    DHCP Server . . . . . : 192.168.1.1
    DHCPv6 IAID . . . . . : 86825735
    DHCPv6 Client DUID. . . . . : 00-01-00-01-30-6F-00-47-2C-08-07-41-7E-95
    DNS Servers . . . . . : 192.0.2.1
    NetBIOS over Tcpip. . . . . : enabled
  
```

Figure 23 (C4): `ipconfig /all` showing DNS server = `192.0.2.1` (the broken setting)

Isolating the layer. Pinging the public address `8.8.8.8` still works, proving that the Wi-Fi link, the gateway and IP routing are fine. Only name resolution is broken.



```

C:\Windows\System32>ping 8.8.8.8 -n 4

Pinging 8.8.8.8 with 32 bytes of data:
Reply from 8.8.8.8: bytes=32 time=63ms TTL=115
Reply from 8.8.8.8: bytes=32 time=28ms TTL=115
Reply from 8.8.8.8: bytes=32 time=30ms TTL=115
Reply from 8.8.8.8: bytes=32 time=29ms TTL=115

Ping statistics for 8.8.8.8:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 28ms, Maximum = 63ms, Average = 37ms
  
```

Figure 24 (C5): Ping `8.8.8.8` works (4/4 replies): the network is fine, only DNS is broken

4.2 Second failure: TCP port blocked

To show a transport-layer failure, I tested the same host on the expected HTTPS port (443) and on an unused port (444), capturing with the filter "`port 443 or port 444`".

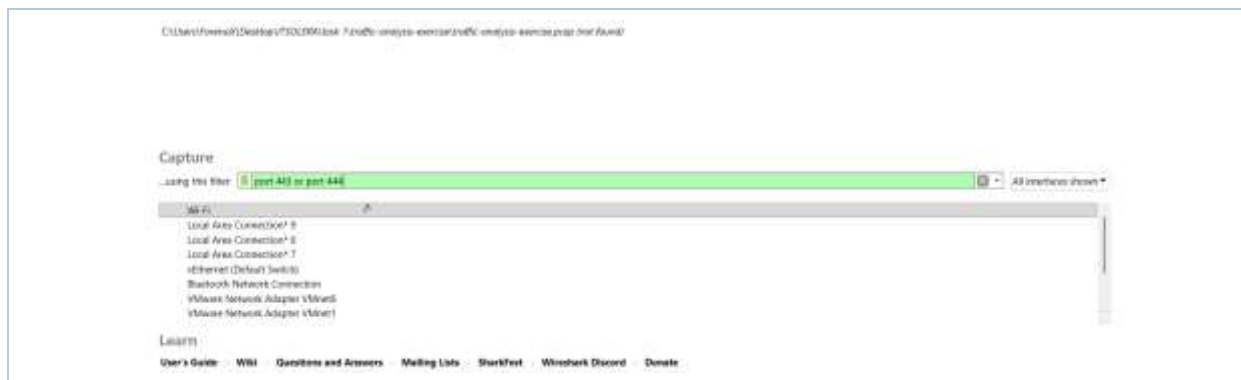


Figure 25 (D0): Wireshark capture setup for the port test (re-created as text): filter covering the working port 443 and the blocked port 444

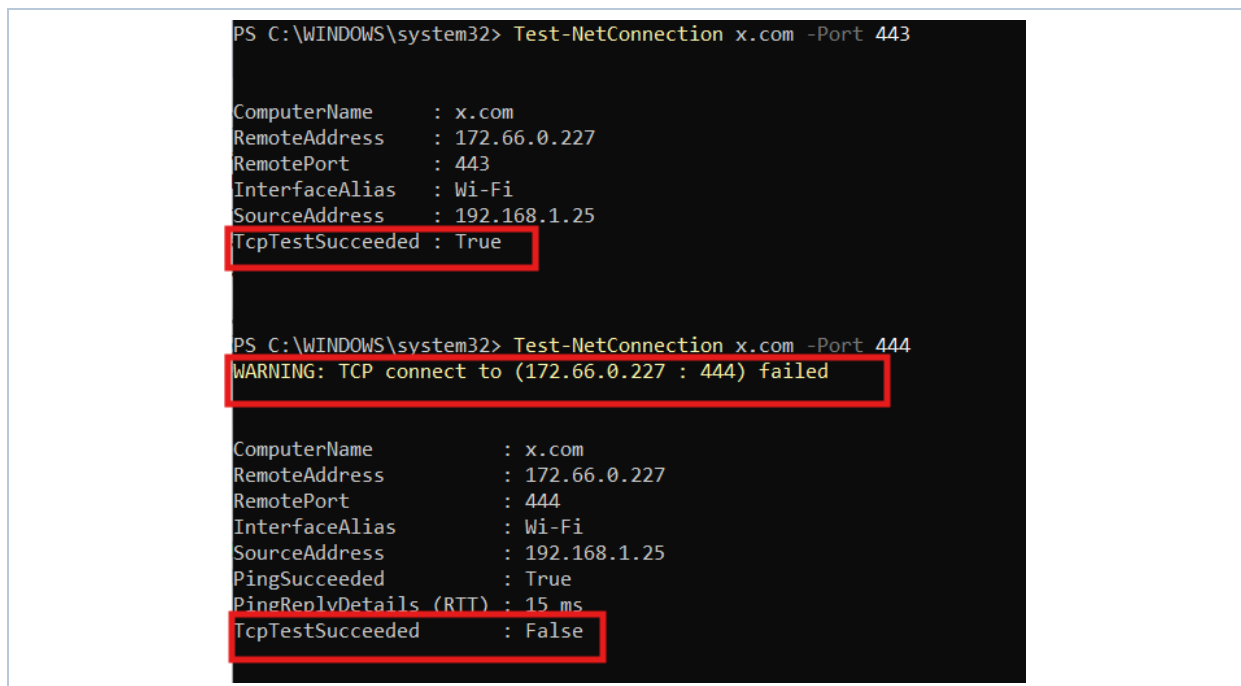


Figure 26 (D1): Test-NetConnection: port 443 succeeds (TcpTestSucceeded True); port 444 fails (TcpTestSucceeded False, PingSucceeded True)

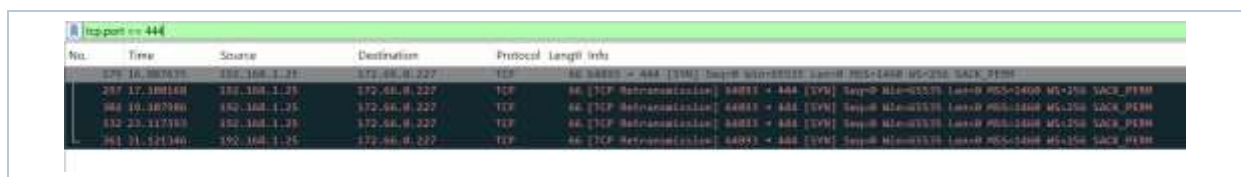


Figure 27 (D2): Port 444 blocked: the SYN is sent and retransmitted 4 times with no SYN-ACK received

Reading the evidence

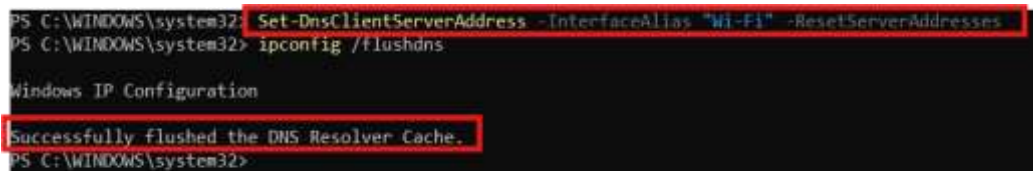
The host answers ping, so the network path and IP layer work. On port 444 the SYN is retransmitted with growing gaps (about 1, 2, 4 and 8 seconds) and never answered. A closed port on a reachable host normally returns a reset, so silence suggests a firewall or filter dropping the packets. Port 443 completes the handshake, so the service itself is healthy.

5 TROUBLESHOOTING PROCEDURE

I followed a logical bottom-up approach: confirm the local configuration, then test reachability outward one layer at a time, and only then look at the application. Each test either clears a layer or points to the fault.

5.1 Step-by-step diagnosis and fix

#	Action / command	Layer tested	Result in my lab
1	Observe and record the symptom	Application	Website will not open; "Could not resolve host" (Figure C2)
2	ping 8.8.8.8	Internet (IP)	4/4 replies: internet connectivity fine (C5)
3	nslookup x.com / ping x.com	Application (DNS)	Name cannot be resolved: fault is name resolution (C2)
4	ipconfig /all	Configuration	DNS server is 192.0.2.1, not the router (C4)
5	Wireshark filter ip.dst == 192.0.2.1	Transport (UDP 53)	Repeated DNS queries, no replies (C3)
6	Conclude root cause	All	Wrong, unreachable DNS server on the client
7	Set-DnsClientServerAddress -ResetServerAddresses; ipconfig /flushdns	Configuration	DNS reset to 192.168.1.1, cache flushed (C6a)
8	nslookup x.com; ping x.com	Application (DNS)	Resolves to 162.159.140.229; 0% loss (C6)
9	Full verification (Section 5.2)	Layers 1 to 4	All tests pass (E1 to E7)
10	Document the fault, cause and fix	Process	This report and team contribution record



```

PS C:\WINDOWS\system32> Set-DnsClientServerAddress -InterfaceAlias "Wi-Fi" -ResetServerAddresses
PS C:\WINDOWS\system32> ipconfig /flushdns

Windows IP Configuration

Successfully flushed the DNS Resolver Cache.
PS C:\WINDOWS\system32>
  
```

Figure 28 (C6a): PowerShell: DNS reset to original (ResetServerAddresses), cache flushed

```

C:\Windows\System32>nslookup x.com
Server: UnKnown
Address: 192.168.1.1

Non-authoritative answer:
Name: x.com
Address: 162.159.140.229

C:\Windows\System32>ping x.com -n 4

Pinging x.com [162.159.140.229] with 32 bytes of data:
Reply from 162.159.140.229: bytes=32 time=16ms TTL=57
Reply from 162.159.140.229: bytes=32 time=23ms TTL=57
Reply from 162.159.140.229: bytes=32 time=15ms TTL=57
Reply from 162.159.140.229: bytes=32 time=20ms TTL=57

Ping statistics for 162.159.140.229:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milliseconds:
        Minimum = 15ms, Maximum = 23ms, Average = 18ms

```

Figure 29 (C6): DNS fixed: nslookup x.com (162.159.140.229) and ping x.com (4/4 replies, 0% loss) working again

5.2 Verification after the fix

After the repair I re-tested every layer from the bottom up to prove that the whole path works, not only DNS.

Test	Command	Layer / what it proves
E1	ipconfig /all	Configuration restored: IP 192.168.1.25, gateway and DNS 192.168.1.1
E2	ping 192.168.1.1	Local network and gateway reachable (0% loss)
E3	ping 8.8.8.8	Internet layer: routing beyond the LAN works
E4	nslookup x.com	Application: DNS resolves x.com again
E5	Test-NetConnection x.com -Port 443	Transport: TCP handshake to port 443 succeeds
E6	tracert -d x.com	Path: 8 hops from the gateway to 172.66.0.227
E7	curl -I https://x.com	Application: HTTP/1.1 200 OK from the web server

```

Wireless LAN adapter Wi-Fi:

Connection-specific DNS Suffix . : 
Description . . . . . : Intel(R) Wi-Fi 6 AX201 160MHz
Physical Address. . . . . : 2C-DB-07-41-7E-95
DHCP Enabled. . . . . : Yes
Autoconfiguration Enabled . . . . : Yes
IPv6 Address. . . . . : 2400:adc7:2104:9900:f027:418e:ac15:976b(Preferred)
Temporary IPv6 Address. . . . . : 2400:adc7:2104:9900:a1d6:17b2:1346:b4db(Preferred)
Link-local IPv6 Address . . . . . : fe80::7ac5:1310:5277:748e%9(Preferred)
IPv4 Address. . . . . : 192.168.1.25(Preferred)
Subnet Mask . . . . . : 255.255.255.0
Lease Obtained. . . . . : Wednesday, September 30, 2026 7:14:26 PM
Lease Expires . . . . . : Thursday, October 1, 2026 7:14:27 PM
Default Gateway . . . . . : fe80::1000%9
                          192.168.1.1
DHCP Server . . . . . : 192.168.1.1
DHCPv6 IAID . . . . . : 86825735
DHCPv6 Client DUID. . . . . : 00-01-00-01-30-6F-00-47-2C-DB-07-41-7E-95
DNS Servers . . . . . : 192.168.1.1
Netbios over Tcpip. . . . . : Enabled

```

Figure 30 (E1): ipconfig /all: Wi-Fi IPv4 192.168.1.25, gateway 192.168.1.1, DNS 192.168.1.1 (restored)

```

C:\Windows\System32>ping 192.168.1.1 -n 4

Pinging 192.168.1.1 with 32 bytes of data:
Reply from 192.168.1.1: bytes=32 time=4ms TTL=64
Reply from 192.168.1.1: bytes=32 time=1ms TTL=64
Reply from 192.168.1.1: bytes=32 time=5ms TTL=64
Reply from 192.168.1.1: bytes=32 time=1ms TTL=64

Ping statistics for 192.168.1.1:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 1ms, Maximum = 5ms, Average = 2ms

```

Figure 31 (E2): Ping default gateway 192.168.1.1 (4/4 replies, 0% loss)

```

C:\Windows\System32>ping 8.8.8.8 -n 4

Pinging 8.8.8.8 with 32 bytes of data:
Reply from 8.8.8.8: bytes=32 time=112ms TTL=115
Reply from 8.8.8.8: bytes=32 time=32ms TTL=115
Reply from 8.8.8.8: bytes=32 time=31ms TTL=115
Reply from 8.8.8.8: bytes=32 time=32ms TTL=115

Ping statistics for 8.8.8.8:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 31ms, Maximum = 112ms, Average = 51ms

```

Figure 32 (E3): Ping 8.8.8.8 (4/4 replies, 0% loss): internet reachable

```

C:\Windows\System32>nslookup x.com
Server: UnKnown
Address: 192.168.1.1

Non-authoritative answer:
Name: x.com
Address: 172.66.0.227

```

Figure 33 (E4): nslookup x.com resolves to 172.66.0.227: DNS working

```

PS C:\WINDOWS\system32> Test-NetConnection x.com -Port 443

ComputerName : x.com
RemoteAddress : 172.66.0.227
RemotePort    : 443
InterfaceAlias : Wi-Fi
SourceAddress  : 192.168.1.25
TcpTestSucceeded : True

```

Figure 34 (E5): Test-NetConnection x.com port 443 (TcpTestSucceeded True)

```

C:\Windows\System32>tracert -d x.com

Tracing route to x.com [172.66.0.227]
over a maximum of 30 hops:

  1    1 ms    1 ms    1 ms  192.168.1.1
  2   35 ms   47 ms   47 ms  202.163.120.254
  3    *      *      4 ms  192.168.133.54
  4   40 ms   26 ms   16 ms  10.15.225.149
  5   17 ms   15 ms   12 ms  192.168.200.205
  6   14 ms   13 ms   14 ms  192.168.200.18
  7   14 ms   14 ms   15 ms  103.213.109.251
  8   15 ms   13 ms   13 ms  172.66.0.227

Trace complete.

```

Figure 35 (E6): tracert -d x.com: 8 hops from the gateway 192.168.1.1 to x.com (172.66.0.227), trace complete

```

C:\Windows\System32>curl.exe -I https://x.com
HTTP/1.1 200 OK
Date: Wed, 30 Sep 2026 18:14:03 GMT
Content-Type: text/html; charset=UTF-8
Connection: keep-alive
Server: cloudflare envoy
x-server: x-web
Cache-Control: no-cache, no-store, max-age=0
referrer-policy: strict-origin-when-cross-origin
x-frame-options: SAMEORIGIN
x-transaction-id: 950c114918aa6636
x-xss-protection: 0
x-download-options: noopen
origin-agent-cluster: ?1
x-content-type-options: nosniff
x-dns-prefetch-control: off

```

Figure 36 (E7): curl -I https://x.com: HTTP/1.1 200 OK, Server: cloudflare envoy

5.3 Symptom-to-cause guide

Symptom	Likely cause	First test
Ping by IP works, by name fails	DNS server wrong or unreachable	nslookup, ipconfig /all
Ping works, one TCP port times out	Firewall or filter on that port	Test-NetConnection -Port
Nothing outside the LAN reachable	Gateway or routing problem	ping gateway, tracert

Symptom	Likely cause	First test
Port reachable but page errors	Application or TLS problem	curl -v, inspect TLS Hello

5.4 Prevention

- Obtain DNS settings automatically from the router (DHCP) and avoid typing addresses by hand.
- Configure a secondary DNS server so one failure does not stop name resolution.
- Keep firewall rules documented so that a blocked port can be recognised quickly.
- Keep a saved packet capture and its hash as evidence when investigating incidents.

6 CONCLUSION

My captures show the two transport protocols working side by side in a single secure website visit: UDP for the quick DNS lookup and TCP for the reliable HTTPS session. The DNS experiment showed that a fault at the application layer can look like a total outage even when TCP and IP are healthy. The blocked-port test showed how a silent, retransmitted SYN separates a filtered port from a healthy one.

The main lessons are to troubleshoot layer by layer, to use the simplest tests first (ipconfig, ping, nslookup), to confirm with a packet capture, and to verify the fix across the whole path. This method is repeatable and suitable for any TCP/IP failure.

7 Contribution declaration

Item	Detail
Assigned role	TCP vs UDP, application examples, communication failure, troubleshooting procedure
Completed work	Packet captures, DNS failure and blocked-port experiments, fix and verification, this report
Supporting evidence	36 evidence figures (Figures 1 to 36) from my own captures and commands, and saved capture file with SHA-256 hash
Portal declaration	To be submitted separately through the portal